

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET
MÉTIER ANGERS

Projet Vision Robotique 2026

PJT 2A S8 2026

COMPTE RENDU

Élèves :

Isam GHARSALLI

Adrien MERIC

Vivian SZEREMENT



Remerciements

Nous remercions les professeurs qui nous ont accompagnés durant ce projet.

Table des matières

I	Feuille de route initiale	4
1	Objet et objectifs du Projet	4
2	Problématique	4
3	Démarches envisagées et résolutions proposées	5
3.1	Le Robot	5
3.2	État de l'art	5
3.3	Cahier des charges provisoire	5
3.4	Outil d'IA intégré : Gemini Cli	6
4	Partenaires mobilisés : Services internes à l'Ecole, interlocuteurs extérieurs	6
II	Planning	7
1	Objectifs	7
2	Analyse des écarts de planning	8
III	Corpus	9
1	Déroulé effectif	9
1.1	Interface logiciel : webcam et suivi du mouvement	9
1.2	Paramétrage des limites du mouvement et sécurité	10
1.3	Code python et traitement des données	11
1.4	Difficultés rencontrées	12
1.5	Guide d'installation	13
IV	Conclusion	14
V	Annexes	15
1	Code DRL	15

Table des figures

I.1	Robot Doosan utilisé pour les manipulations	4
II.1	Gantt chart initial	7
II.2	Planning et Gantt Chart finaux	8
III.1	Interface de vision et reconnaissance d'un objet circulaire rouge	9
III.2	Reconnaissance de la paume de la main	9
III.3	Configurations du robot en fonction des contraintes posées sur les axes . .	10
III.4	Démonstration de la fonction "Dance"	11
III.5	Illustration guide installation	13

Liste des tableaux

1	Planning du Projet Vision Robotique	7
---	---	---

Listings

- 1 Script DRL (Python) pour le suivi absolu du cobot M0609 15

I Feuille de route initiale

1 Objet et objectifs du Projet

Le projet s'intègre dans une dynamique plus large de développement de la robotique aux Arts et Métiers. Il servira comme démonstrateur notamment à un forum et éventuellement lors de journée portes ouvertes. Ce projet sera également présenté à l'entreprise Alstom dans le cadre d'un concours de robotique.

L'objectif principal est de réaliser un robot mime interactif qui soit adapté à une activité avec des enfants. Un opérateur prend un objet dans une main et on souhaite que le robot suive en temps réel le déplacement de l'objet et reproduise le mouvement. Le robot devra respecter des normes de sécurité strictes qui seront développées ci-après.



FIGURE I.1 – Robot Doosan utilisé pour les manipulations

2 Problématique

Comment piloter de manière robuste un robot collaboratif pour qu'il soit interactif avec le public ?

3 Démarches envisagées et résolutions proposées

Nous allons d'abord étudier le robot et la façon dont il est utilisé dans son environnement afin d'établir un cahier des charges.

3.1 Le Robot

Le robot que nous utilisons lors de ce projet est un robot M069 de l'entreprise Doosan Robotics. Ce robot est conçu pour manipuler des charges lourdes avec une grande précision, grâce à sa portée de 1700 mm et à sa structure robuste. Il est particulièrement adapté aux environnements industriels de grande taille où il peut automatiser des tâches de production.

Son principal usage est la collaboration homme-robot car il intègre des systèmes de sécurité comme la détection de couple. Il sert donc à améliorer la productivité tout en garantissant la sécurité des opérateurs et s'intègre facilement dans des lignes de production existantes grâce à son interface intuitive.

3.2 État de l'art

Pour pouvoir contrôler le robot nous allons utiliser le logiciel DRL Studio. Ce logiciel permet de programmer et contrôler les mouvements du robot H2017 à l'aide du langage DRL. Il sert aussi à simuler et tester les tâches du robot avant leur utilisation réelle afin d'optimiser la production et éviter les erreurs. Le logiciel Dart fourni par Doosan est aussi disponible et nous permet d'obtenir des informations comme le centre de masse et la masse totale du robot.

Plusieurs bibliothèques Python sont disponibles, dont DoosanV1, que nous utiliserons pour contrôler notre robot. Le dépôt doosan-python disponible sur GitHub facilitera l'utilisation et l'intégration de cette bibliothèque.

Nous allons utiliser les bibliothèques openCV et mediapipe qui nous permettent de modéliser des mouvements complexes à l'aide d'une caméra.

La première partie se concentrera principalement sur l'activation de la caméra et sur la reconnaissance d'une forme, par exemple une balle d'une couleur bien choisie qui servira de repère à suivre pour le robot. Pour cela, nous utiliserons mediapipe afin de détecter et reconnaître ces formes. Cette partie sera exécutée sur un IDE différent de DRL Studio. La seconde partie du programme traitera des différents mouvements du robot. Ceux-ci devront s'effectuer dans un plan normal à celui de la caméra. Cette partie sera exécutée sous DRL Studio.

Les axes d'amélioration passeront principalement par l'optimisation des programmes de reconnaissance et des programmes contrôlant les mouvements du cobot.

3.3 Cahier des charges provisoire

On propose un cahier des charges provisoire à l'issu de la première séance. On souhaite faire une démonstration percutante pour apporter un côté ludique à la technologie.

- Implémenter un algorithme python de base pour assurer le squelette du projet. Il devra permettre au robot de suivre la trajectoire d'une balle.
 - 2D : le robot doit pouvoir suivre la balle dans un plan perpendiculaire à la surface du sol, qui contient la balle.
- 3D : le robot doit pouvoir se déplacer dans un plan orthogonal au plan de la surface du sol, qui contient la balle.
- Le robot ne doit pas dépasser la vitesse en translation maximale admissible.
 - Le robot ne doit pas dépasser la vitesse angulaire maximale admissible.
 - Le robot devra être capable de différencier différentes couleurs de balle et ne suivre que celle qu'on lui demande (on pourra prendre deux balles de couleurs différentes et les agiter en même temps, le robot doit suivre la bonne)
 - Robot danseur : quoi de plus amusant et démonstratif qu'un robot danseur. Un mouvement particulier et reconnaissable de la main doit pouvoir faire démarrer une danse du robot en actionnant différents moteurs de façon à avoir un mouvement fluide et reconnaissable. L'utilisation de la bibliothèque médiapipe permet de telles actions. On pourra penser à étoffer le panel d'actions du robot en fonction des différents signes de main reconnus par médiapipe.

3.4 Outil d'IA intégré : Gemini Cli

Pour aider au codage et proposer des squelettes de codes efficaces, on utilise un outil d'IA intégré de Google : Gemini Cli. Cet outil extrêmement efficace est installé au niveau de l'ordinateur lui-même et agit sur la machine en fonction du niveau d'accès qu'on souhaite lui donner. Dans notre cas, il a été utilisé directement dans la console pour pouvoir modifier le code python. L'intelligence artificielle est utilisée ici dans un but pédagogique ; elle permet d'établir un squelette de projet et de déboguer certaines parties de notre code mais n'est pas capable de répondre à tous nos besoins.

4 Partenaires mobilisés : Services internes à l'Ecole, interlocuteurs extérieurs

Enseignants mobilisés : Julien ARTOZOUL, Germain GUENAEL

II Planning

1 Objectifs

Séance	Phase	Objectifs et Jalons
S1	Cadrage	Définition du cahier des charges, analyse des précédents rapports et segmentation du projet.
<i>AVANT S2</i>	<i>Infrastructure</i>	Jalon : prendre RDV avec les professeurs, Setup Python, Git, et installation des logiciels pour robot.
S2 – S3	Perception	Prise en main robot. Développement et tests des algorithmes de vision (OpenCV/IA).
S4 – S5	Intégration	Implémentation de l’asservissement visuel et corrections algorithmiques.
S6 – S7	Évolution	Extension du cahier des charges. Implémentation de fonctionnalités avancées.
S8 – S9	Finalisation	Tests, corrections finales et préparation de la soutenance.
S10	Validation	Soutenance : Démonstration technique et analyse des résultats.

TABLE 1 – Planning du Projet Vision Robotique

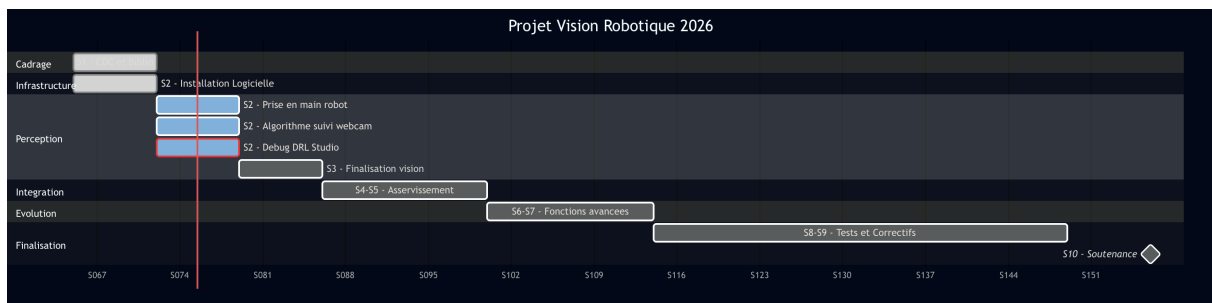


FIGURE II.1 – Gantt chart initial

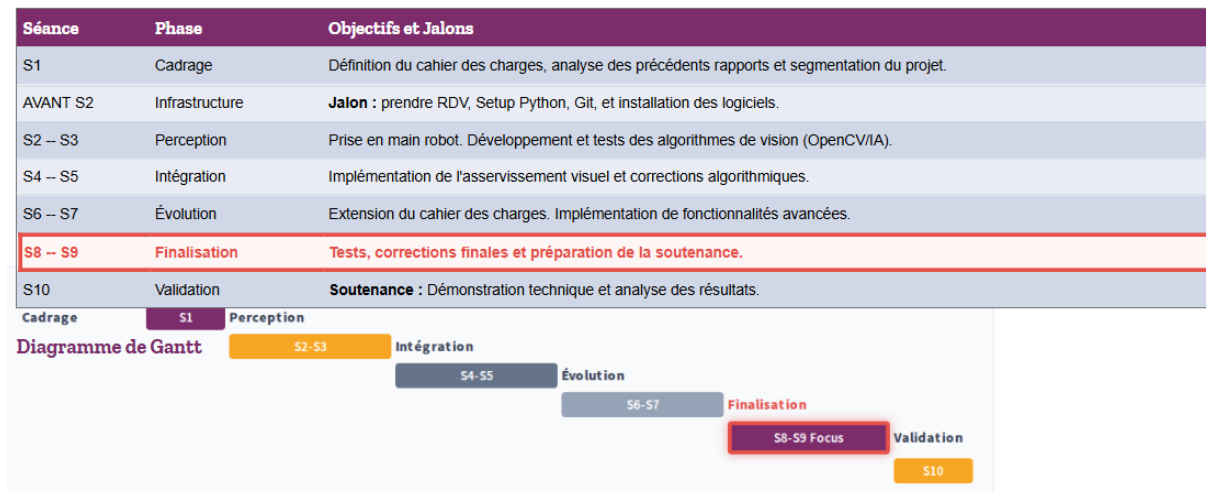


FIGURE II.2 – Planning et Gantt Chart finaux

2 Analyse des écarts de planning

Le déroulement du projet s'est globalement conformé au diagramme de Gantt initial. Les phases de perception (S2-S3) et d'intégration de l'asservissement (S4-S5) ont été tenues. Néanmoins, le temps alloué à la phase des fonctions avancées (S6-S7) a dû être complété lors des séances 8 et 9 pour l'implémentation d'une fonction danse. Lors de la séance 8 nous avons corrigé les instabilités dynamiques (vibrations) et nous avons rédigé un guide d'installation.

III Corpus

1 Déroulé effectif

1.1 Interface logiciel : webcam et suivi du mouvement

A l'issue des premières séances, nous avons été capables de développer l'interface de vision pour le robot. Le code permet d'ouvrir la caméra de l'ordinateur et de reconnaître la couleur demandée, rouge, bleu ou vert, comme on peut le voir sur la figure III.1. C'est cette interface qui va être à la base du projet. Le programme va permettre de créer un barycentre pour l'objet de couleur reconnue et convertir la position lue pour transmettre au robot.

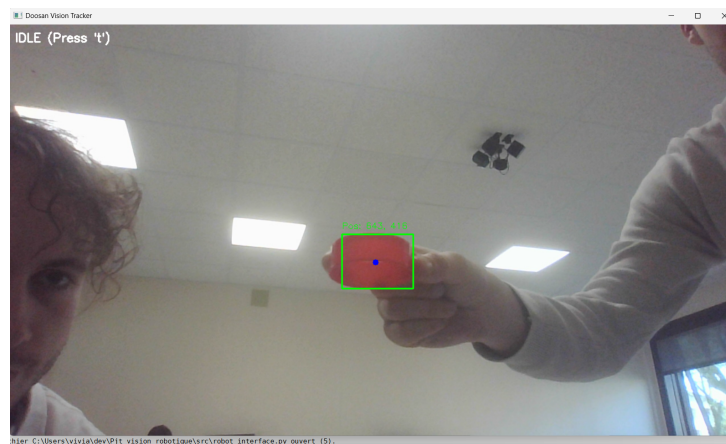


FIGURE III.1 – Interface de vision et reconnaissance d'un objet circulaire rouge

Nous avons également adapté le programme pour reconnaître les mains d'un opérateur à l'aide de la bibliothèque médiapipe de Google. A ce stade, il est capable de reconnaître les doigts séparément, la paume de la main, le symbole "peace", etc... Un exemple du symbole palm est visible en figure III.2

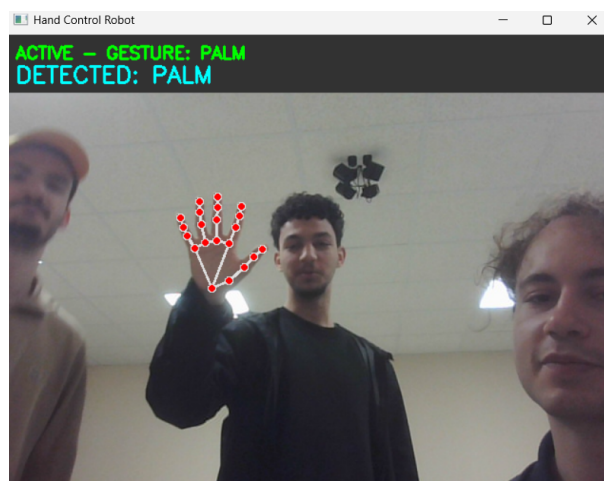


FIGURE III.2 – Reconnaissance de la paume de la main

1.2 Paramétrage des limites du mouvement et sécurité

Une partie fondamentale de ce projet consiste à limiter les mouvements du robot pour éviter des blocages et mises en sécurité intempestifs à chaque manipulation. A cause de l'inertie du robot et de l'accélération du mouvement, ces blocages arrivent très rapidement et le processus de remise en route est long et fastidieux. Pour éviter ça, on va donc borner cette accélération, la vitesse qui en découle et placer des limites sur les angles admissibles sur chaque axes. Le robot est piloté en cartésien et pas dans la base des axes (J1, ..., J6). On pose des limites sur ces axes. On observe le résultat en figure III-3.



(a) Mise en sécurité du robot (liseré rouge) en l'absence de contraintes. Nécessite de redémarrer l'ensemble.



(b) Le robot s'arrête avant la mise en sécurité mais reste sous notre contrôle (liseré blanc)

FIGURE III.3 – Configurations du robot en fonction des contraintes posées sur les axes

Afin de pallier à l'inertie importante du cobot M0609, des limites logicielles strictes régissant les accélérations et les vitesses admissibles ont été implémentées. Pour éviter les singularités et garantir un mouvement dans un plan sécurisé, le pilotage s'effectue en coordonnées absolues cartésiennes. Les limites de l'espace de travail sont vérifiées de manière continue par la fonction `check-position-condition` sur les axes X et Z (-350 mm à 350 mm). Si la cible est inatteignable ou hors tolérance (vérification via `check-motion`), le système bascule dans un état de sécurité et ordonne un retour automatique à l'origine spatiale.

1.3 Code python et traitement des données

Pour contrôler le robot et effectuer le traitement de données, nous utilisons le langage python et les différentes bibliothèques présentées plus tôt dans ce projet. Comme indiqué précédemment, nous utilisons deux IDE python, un pour le traitement de données et un pour le contrôle du robot. L'entiereté des codes est disponibles en annexe 1.

L'architecture logicielle repose sur une communication client-serveur TCP/IP opérant sur le port 20002.

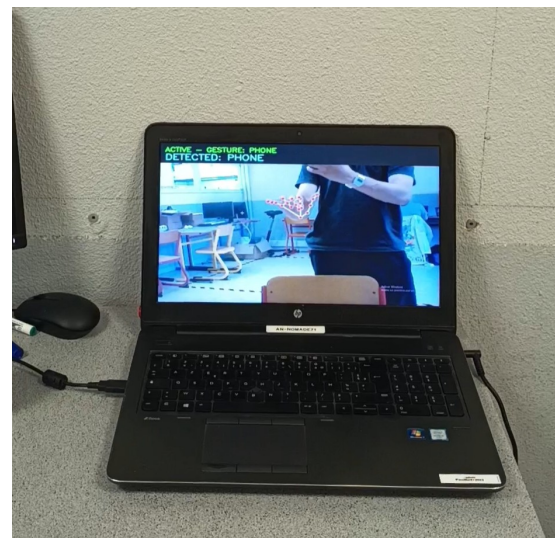
Côté vision (Client Python), l'acquisition du flux vidéo est gérée par OpenCV, tandis que l'extraction des points d'intérêt (landmarks) de la main est assurée par le réseau de neurones de Mediapipe.

L'algorithme isole les coordonnées de la paume (point MCP du majeur) pour définir un barycentre dynamique. Les gestes reconnus ("PALM", "FIST", "PHONE") sont traduits en commandes spécifiques, comme le suivi de trajectoire, le retour à l'origine, ou le déclenchement d'une routine de danse.

Côté contrôle (Serveur DRL), le robot initialise sa position absolue au démarrage. Il interprète les offsets 2D reçus depuis la caméra pour calculer une cible cartésienne, puis utilise la commande amovel pour générer un asservissement fluide et continu.



(a) Chorégraphie de la fonction "Dance" effectué par le robot



(b) Signe "PHONE" de la main reconnue

FIGURE III.4 – Démonstration de la fonction "Dance"

Dans le cas où le système interprète le signe "PHONE", le programme envoie une consigne sous forme de texte au lieu de transmettre des coordonnées. Le code exécuté sur le DRL lit alors cette instruction, par exemple "Dancing Time", puis déclenche l'exécution

d'une chorégraphie pendant une durée déterminée. À l'issue de cette séquence, un temps de pause d'une minute est appliqué durant lequel le robot ne peut pas repasser en mode danse, même s'il reçoit de nouveaux ordres.

1.4 Difficultés rencontrées

Ce projet s'inscrit dans la continuité d'autres projets de robotique réalisés au sein de l'école et est destiné à servir aux futurs étudiants des Arts et Métiers pour être amélioré et servir de base pour d'autres projets. Dans cette optique, il est important de parler des nombreuses difficultés rencontrées lors de l'élaboration du code, de la structure globale, ainsi que des problèmes matériels récurrents.

Un des problèmes majeur est que la tablette est très lente. Si nous bloquons le robot nous devons la redémarrer ce qui prends 5 à 10 min à chaque fois, c'est donc une perte de temps considérable. Nous avons donc chercher à poser des limites inférieures à celles déjà inscrits sur le robot.

Nous nous sommes appuyés sur l'outil de code Gemini CLI qui nous a permis d'avancer beaucoup plus vite. Cependant le robot a certaines spécificités. Le robot est commandé en python 3.9 ; nous devons donc adapter notre script DRL pour minimiser le code sur DRL et maximiser le code "extérieur".

On peut également communiquer avec le robot avec seulement un seul port, le 20002. l'IA n'arrivait pas à trouver l'erreur, nous avons donc veillé à utiliser le manuel de python de Doosan propre au robot.

L'outil IA nous a également proposé d'utiliser un asservissement en relatif, ce qui a causé un phénomène de dérive. Nous avons donc dû coder à la main un positionnement en absolu.

Nous nous sommes également rendu compte que le robot avait tendance à vibrer puisque quand on bouge lentement notre main le robot tente de faire des micros mouvements.

1.5 Guide d'installation

Le programme a été installé sur un PC de l'ensam. Il se lance cependant d'une manière précise et rigoureuse. Nous avons donc réalisé un guide d'installation et de lancement du projet.

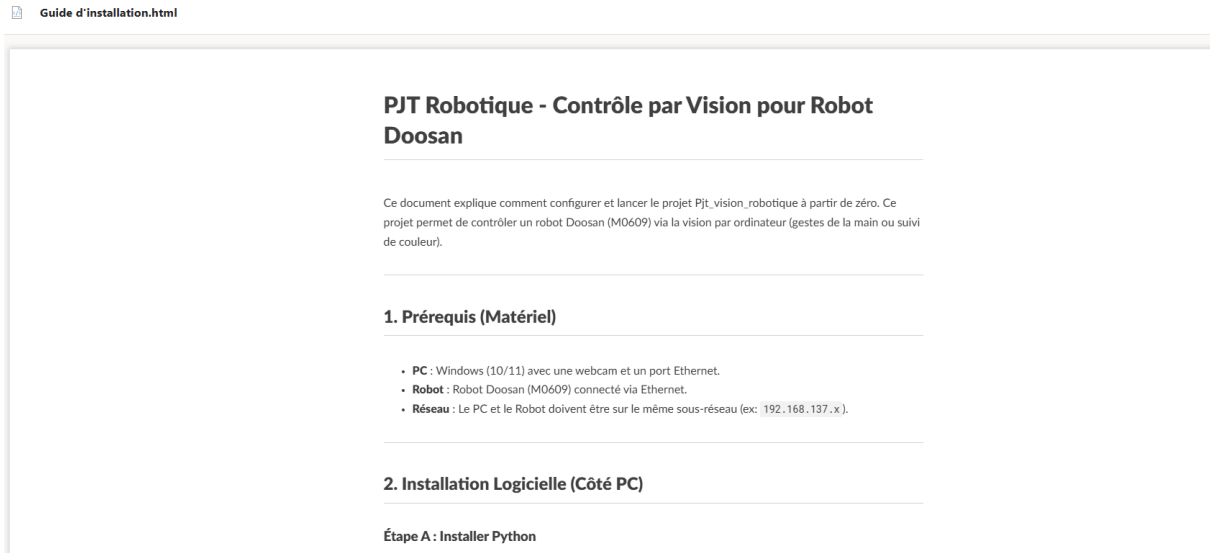


FIGURE III.5 – Illustration guide installation

IV Conclusion

Le projet Vision Robotique 2026 s'est achevé sur un bilan positif, nous avons réussi à atteindre la majorité des objectifs définis lors de la phase de cadrage. Malgré les défis techniques liée à la manipulation d'un cobot industriel, la démonstration de l'utilisation de ce cobot est aujourd'hui fonctionnelle.

Nous avons développé avec succès une interface capable de suivre en temps réel le déplacement d'un objet ou de la main d'un opérateur grâce à l'intégration des bibliothèques OpenCV et Mediapipe. Le robot peut désormais reconnaître des couleurs spécifiques et des gestes complexes, tels que la paume de la main ou le signe "peace", pour les traduire en commandes de mouvement. Un travail conséquent a également été effectué sur le paramétrage des limites de vitesse et d'accélération, permettant au robot de fonctionner en toute sécurité sans subir de blocages dus à l'inertie.

Enfin, bien que le planning ait été ajusté pour finaliser les fonctions avancées, l'implémentation de la chorégraphie de danse déclenchée par le signe "phone" a été menée à bien, offrant ainsi le côté ludique recherché pour les futures présentations de l'école. Ce projet pourra certainement constituer une base solide pour les futurs étudiants voulant prendre en main ce robot afin de mener à bien leur projet.

V Annexes

1 Code DRL

```

1 from DRCF import *
2 import time
3 # Doosan Robot Language (DRL) Script for M0609
4 # Purpose: Absolute Vision Tracking with Safety Fallback to Origin
5
6
7 # 1. INITIALIZATION: Capture Origin (Safe Home)
8 # The robot captures its current position at start-up.
9 # All incoming coordinates from Python will be treated as offsets
10 # relative to this FIXED origin, effectively creating absolute targets.
11 p_origin, _ = get_current_posx()
12 j_origin = get_current_posj()
13 Posj_dance = [posj(-85,30,95,-3,-55,25),
14               posj(-85,30,95,-3,-55,-25),
15               posj(-85,30,95,-3,-55,25),
16               posj(-85,30,95,-3,-55,-25)]
17
18
19 Cond1 = check_position_condition(DR_AXIS_X, min = -350, max = 350)
20 Cond2 = check_position_condition(DR_AXIS_Z, min = -350, max = 350)
21
22 # Set default motion parameters
23 set_velj(30)
24 set_accj(60)
25
26 # Open TCP Server on Port 20002
27 server_sock = server_socket_open(20002)
28
29 t = time.time()
30 a = 0
31
32 while True:
33     # Receive data from Python
34     res, data = server_socket_read(server_sock, 100)
35
36     if res > 0:
37         msg = data.decode('utf-8')
38         lines = msg.split("\n")
39         # Process only the most recent command
40
41         for line in reversed(lines):
42
43             if line == "Dancing time" and a == 0:
44                 a = 1
45                 movej(posj(-90.5, 24.0, 56.5, -8.9, 60.6, 0.4), vel=100,
46 acc=100)
47                 Tinit = time.time()
48                 while time.time() - Tinit < 12:
49                     for P in Posj_dance:
50                         movej(P, vel=180, acc=120)
51                     wait(2)
52                 t = time.time()

```



```

52         movej(posj(-90.5, 24.0, 56.5, -8.9, 60.6, 0.4),vel=100,
acc=100)
53         break
54
55     elif time.time() - t >= 60 and a == 1:
56         a = 0
57
58     elif "," in line:
59         points = line.split(",")
60         try:
61             # Offsets provided by the Vision system
62             offset_x = float(points[0])
63             offset_y = float(points[1])
64
65             # 2. CALCULATE ABSOLUTE TARGET
66             # target = origin + vision_offset
67             # Mapping: offset_x -> Robot X, offset_y -> Robot Z
68             target_pos = posx(p_origin[0] - offset_x,
69                             p_origin[1],
70                             p_origin[2] - offset_y,
71                             p_origin[3],
72                             p_origin[4],
73                             p_origin[5])
74
75             # 3. SAFETY CHECK: Reachability & Limitsq
76             # check_motion(target) returns 0 if the robot can
safely reach the point
77             if Cond1 == True or Cond2 == True:
78                 # Smooth tracking move
79                 amovel(target_pos, vel=200, acc=400)
80             else:
81                 # 4. FALLBACK: Return to Origin
82                 # Triggered if target is out of bounds or near
singularity
83                 tp_log("Target Unreachable! Safety Reset to
Origin.")
84                 movej(j_origin, vel=30, acc=60)
85                 wait(1.0) # Stabilize
86
87                 break # Success, skip older packets
88         except:
89             continue
90
91 server_socket_close(server_sock)

```

Listing 1 – Script DRL (Python) pour le suivi absolu du cobot M0609